

07

线性方程组

7.1 二元一次方程组

从鸡兔同笼到矩阵形式：代数、几何、矩阵三种视角看线性方程组



鸢尾花书·《线性代数不难》 | 开源代码与书稿: <https://github.com/Visualize-ML>

本节你将掌握的核心技能

1 把鸡兔同笼这类实际问题，转化为二元一次方程组

3 几何视角：每个二元一次方程对应一条直线，解为直线的交点

5 矩阵形式 $Ax=b$ ：简洁、高效地表达线性方程组

7 理解三种初等行变换：行交换、行数乘、行倍加

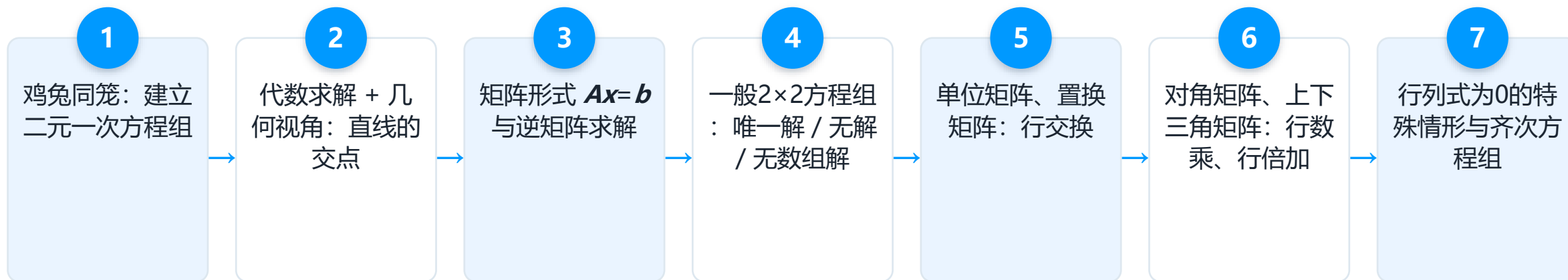
2 代数方法：用代入消元、逆矩阵两种方式求解未知数

4 图解判断方程组的解：唯一解、无解、还是无数组解

6 用行列式 $\det(A)$ 判断方程组是否存在唯一解

8 用 `numpy.linalg.inv()`、`numpy.linalg.solve()` 编程求解方程组

从鸡兔同笼出发，走向矩阵形式的通用解法



《孙子算经》：鸡兔同笼问题

“今有雉兔同笼，上有三十五头，下有九十四足，问雉兔各几何？”

设鸡的数量为 x_1 ，兔的数量为 x_2 ：头数、脚数分别对应一个二元一次方程，两个方程结合成一个线性方程组

二元一次方程 (linear equation in two variables)

$$a_1x_1 + a_2x_2 = b$$

- 含两个未知数 x_1 、 x_2 ，未知数指数最高为1的代数方程； a_1 、 a_2 为任意实系数， b 为任意实数常数
- 头数方程： $x_1 + x_2 = 35$ ；脚数方程： $2x_1 + 4x_2 = 94$ —— 两个二元一次方程结合，构成线性方程组 (system of linear equations)

代入消元法求解鸡兔同笼问题

$$x_1 + x_2 = 35, \quad 2x_1 + 4x_2 = 94 \Rightarrow x_1 = 23, \quad x_2 = 12$$

- 脚数方程两边同除以2，得 $x_1 + 2x_2 = 47$ ；再减去头数方程 $x_1 + x_2 = 35$ ，消去 x_1 ，解得 $x_2 = 12$ ，代回得 $x_1 = 23$
- 笼子里有23只鸡、12只兔——隐含条件： x_1 、 x_2 都是非负整数

图1 • 系数不同，直线的形态各异

- $a_2=0$ 时是竖直线; $a_1=0$ 时是水平线; a_1 、 a_2 均非0时是斜率不为零的斜线

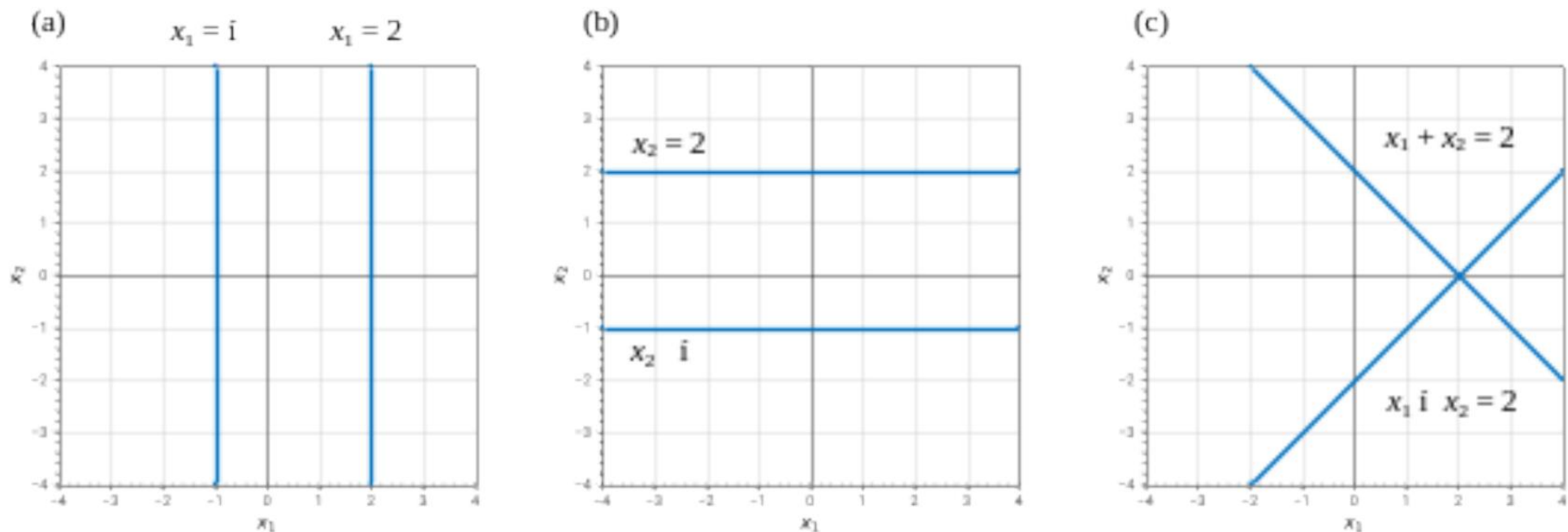


图 1. 几何视角看二元一次方程

图2 • 两条直线的交点就是方程组的解

- 红色线 $x_1 + x_2 = 35$ 、蓝色线 $2x_1 + 4x_2 = 94$ ，交点(23, 12)正是鸡兔同笼问题的解

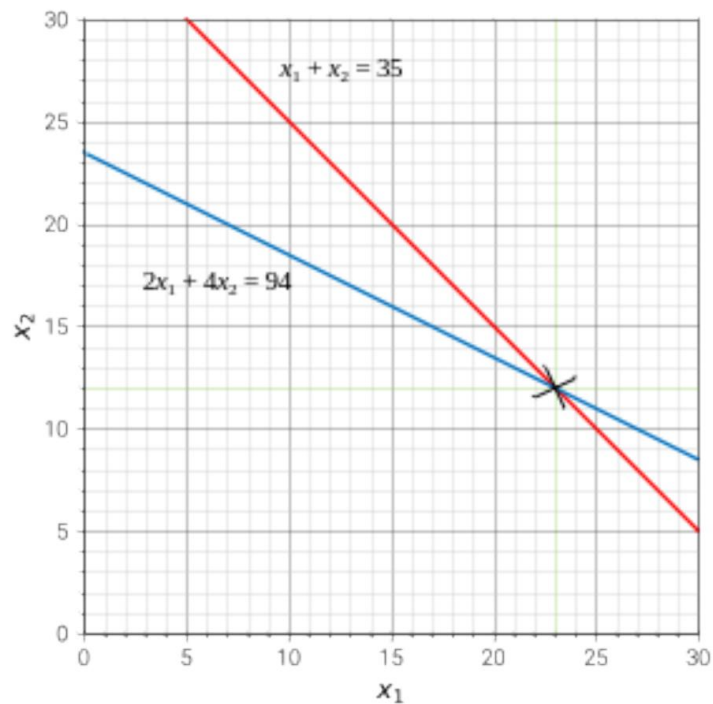


图 2. 图解鸡兔同笼问题

图3 • 把线性方程组写成 $Ax=b$

- 橙色是系数、粉色是未知数、蓝色是常数——两个方程堆叠，(c)就是矩阵形式 $Ax=b$

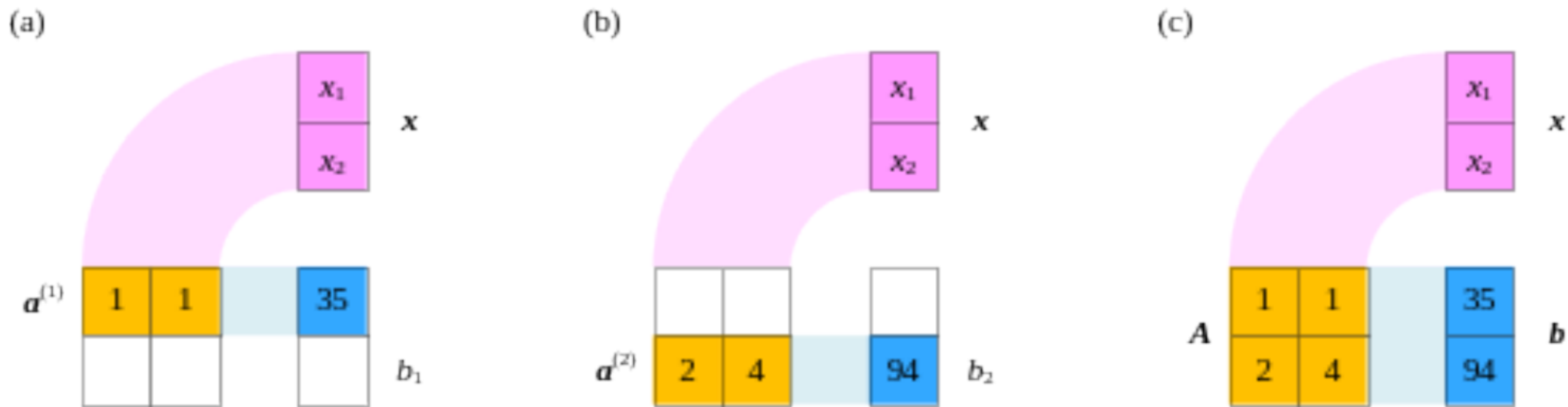


图 3. 把线性方程组写成矩阵乘法形式

$Ax=b$ 的解: $x=A^{-1}b$

$$x = A^{-1}b$$

- 系数矩阵 **A** 叫coefficient matrix, **b** 叫vector of constant terms; **A** 方阵且可逆时, **x** 可由上式直接求得
- 本例中 $\det(A)=2 \neq 0$, **A** 可逆; 代入具体数值, 解得 **$x=[23, 12]^T$** , 与代入消元结果一致

逆矩阵求解线性方程组

用 `numpy.linalg.inv()` 求逆矩阵, 再用 `@` 完成矩阵乘法

- `A = np.array(...)`: 系数矩阵, 每列对应一个未知数, 每行对应一个方程
- `b = np.array(...)`: 常数列向量, 两个元素分别是头数、脚数
- `A_inv = np.linalg.inv(A)`: 求 A 的逆矩阵 (建议先用 `np.linalg.det()` 确认可逆)
- `A_inv @ A` 与 `A @ A_inv`: 验证矩阵与其逆的乘积均为单位矩阵
- `A_inv @ b`: 即为解 x , 也就是鸡、兔的数量

```
## 初始化
import numpy as np

## 鸡兔同笼系数矩阵
A = np.array([[1, 1],
              [2, 4]])

## 常数列向量
b = np.array([[35],
              [94]])

## 矩阵A的逆
A_inv = np.linalg.inv(A)

# 验证
A_inv @ A
A @ A_inv

## 求解
A_inv @ b
```

为什么用矩阵乘法表示线性方程组？

更紧凑：避免反复列出所有变量，变量越多，优势越明显

信息更丰富：分析 A 的秩，可迅速判断唯一解、无解还是无数组解

可推广：超定方程组求解、特征值分解、线性变换等都建立在矩阵表示之上

计算更高效：矩阵运算在计算机中远比逐个求解方程更快、更精确

图4 • 唯一解、无解、无数组解

- (a)两直线重合→无数组解；(b)两直线平行→无解；(c)两直线相交→唯一解

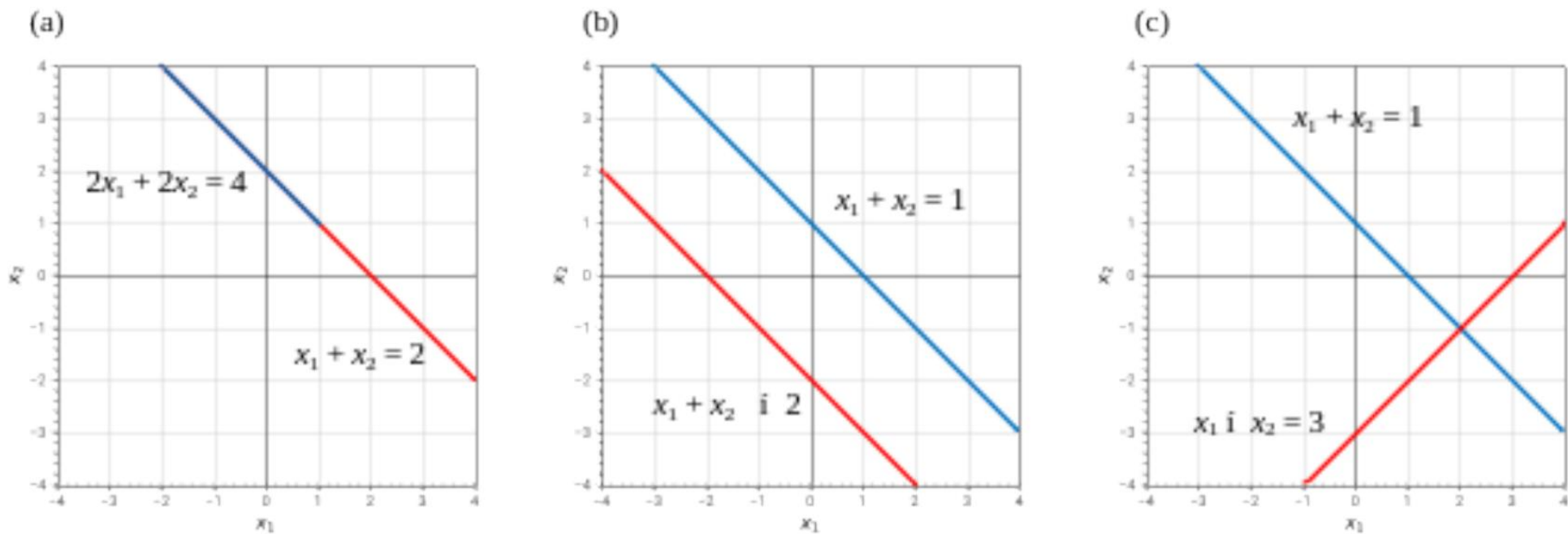


图 4. 不同解的数量

$\det(\mathbf{A})$ 与方程组解的情况

$\det(\mathbf{A})=0 \Rightarrow$ 无解或无数组解; $\det(\mathbf{A})\neq 0 \Rightarrow$ 唯一解

- 图4(a): 两方程等价, $\det(\mathbf{A})=0$, 无数组解; 图4(b): 两直线平行无交点, $\det(\mathbf{A})=0$, 无解
- 图4(c): 两直线相交, $\det(\mathbf{A})=-2\neq 0$, 方阵 $\mathbf{A}$ 可逆, 方程组有唯一解

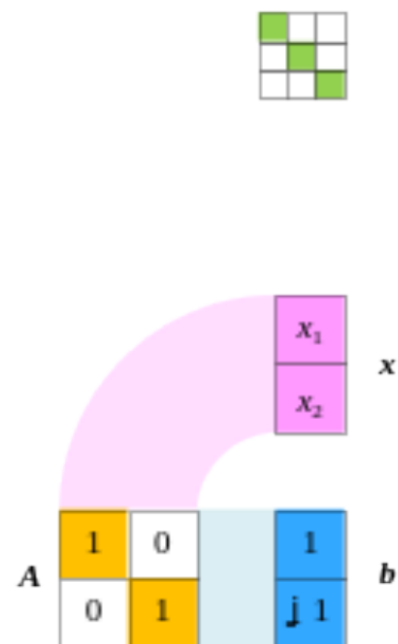
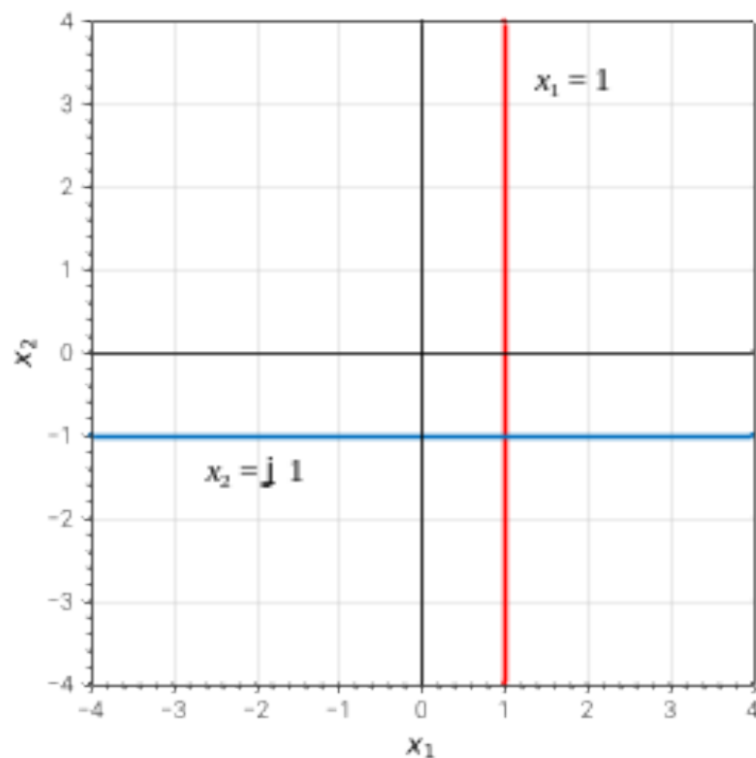
三种初等行变换 (elementary row operations)

- ① 行交换 (row swapping): 交换矩阵的两行
- ② 行数乘 (row scaling): 某一行整体乘以一个非零标量
- ③ 行倍加 (row sum): 一行的非零倍数加到另一行

A 是单位矩阵

图5 • 单位矩阵最“完美”

- $x_1=1$ 、 $x_2=-1$ 直接就是方程组的解，不需要任何额外运算



单位矩阵：无需运算

$A=I$ 时, $x=b$

$$A=I \Rightarrow x=A^{-1}b=b$$

- 求解 $Ax=b$ 的目标, 就是想办法把方阵 A 整理成单位矩阵 I

A 是置换矩阵

图6 • 交换行， A 变为置换矩阵

- $A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, $\det(A) = -1$; 左乘 x 的效果就是“行交换”

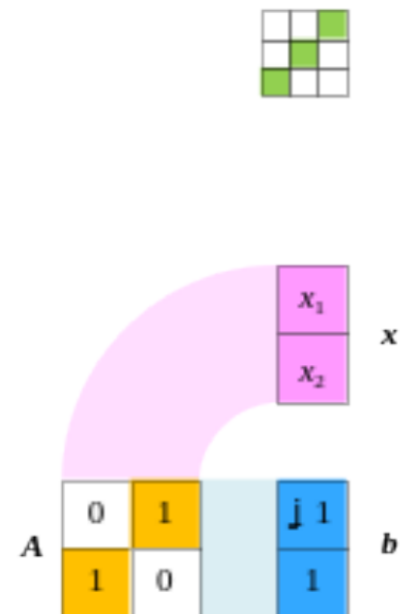
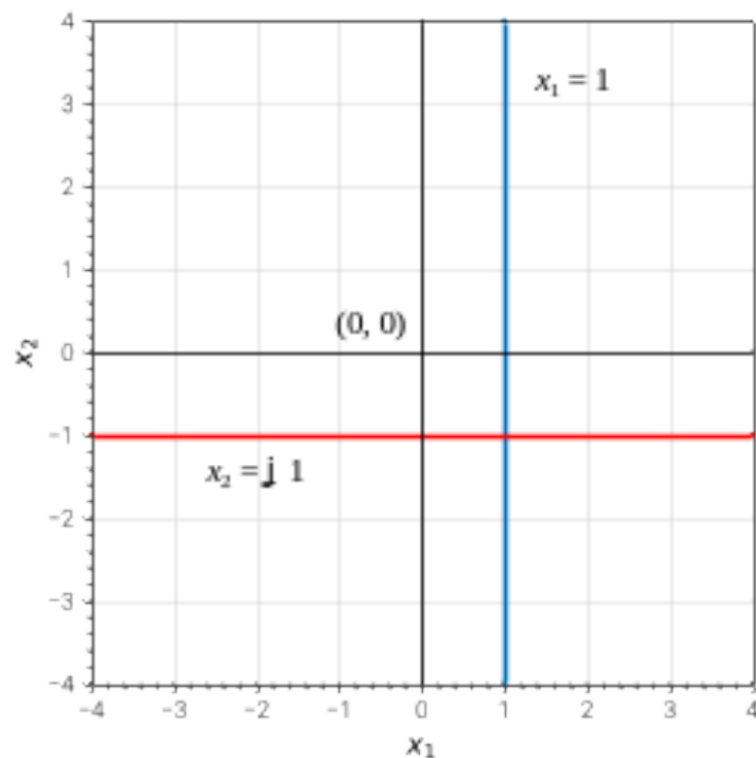


图7 · 行交换只改变直线出现的顺序

- 记作 $R_1 \leftrightarrow R_2$; 行交换 (row swapping) 不影响方程组的解, 两条直线本身没有变化

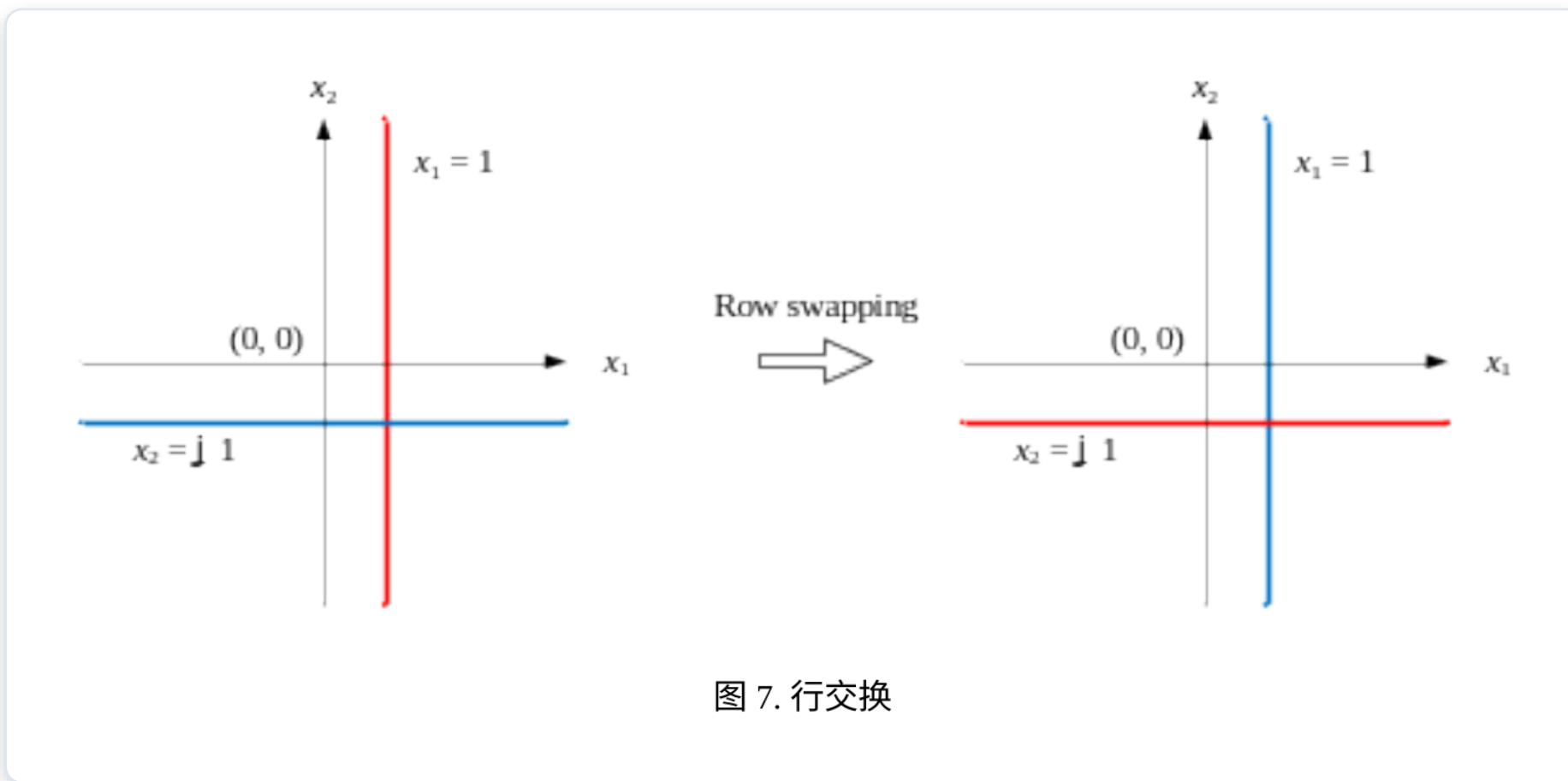


图 7. 行交换

A 是对角方阵

图8 • 对角矩阵已经很接近解

- $A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$, $\det(A) = 6$, 可逆; $2x_1 = 2$ 看起来已经很接近 $x_1 = 1$ 的解

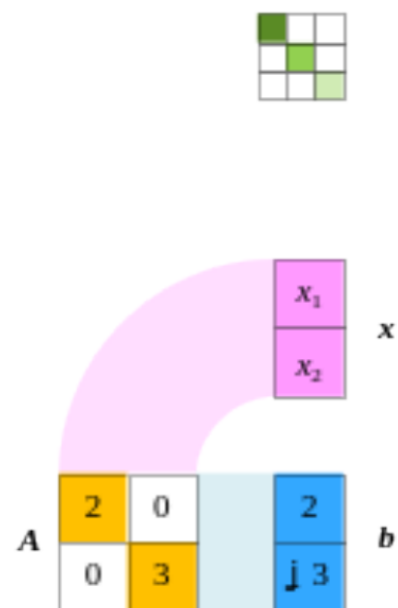
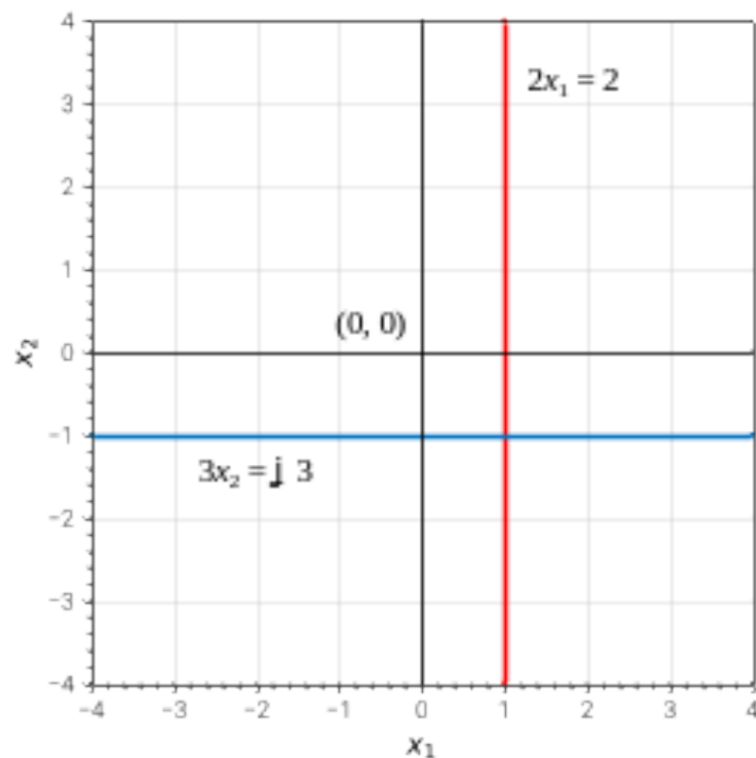


图9 · 行数乘不改变直线的位置、形状

- 记作 $R_1/2 \rightarrow R_1$ 、 $R_2/3 \rightarrow R_2$ ；这两步运算正“藏”在 A 的逆矩阵 A^{-1} 里的 $1/2$ 和 $1/3$ 之中

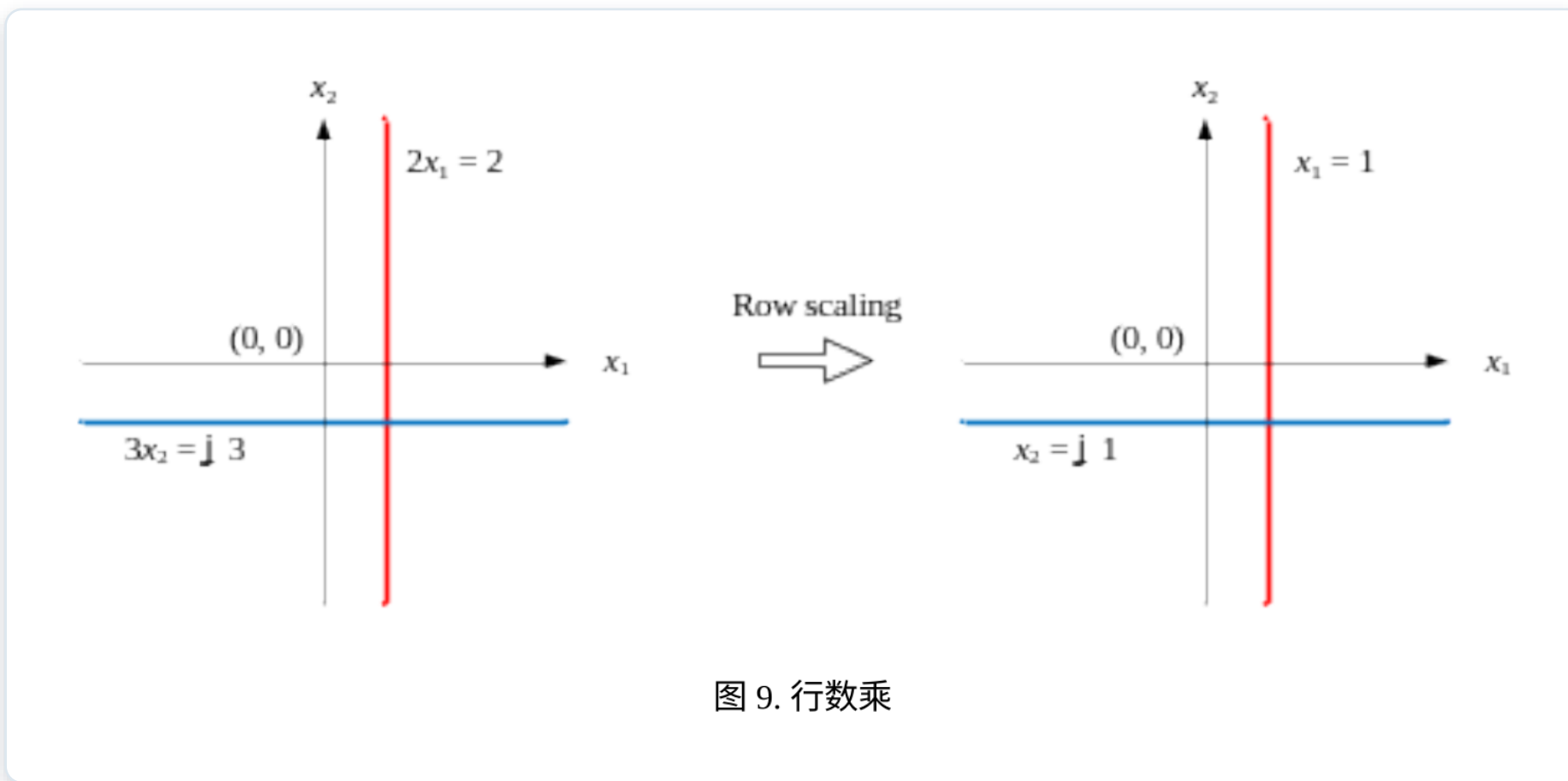


图 9. 行数乘

A 是上三角矩阵

图10 • 上三角矩阵，逐步化简

- $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$, $\det(A) = 1$, 可逆; 第二行 R_2 已经直接给出 x_2 的解

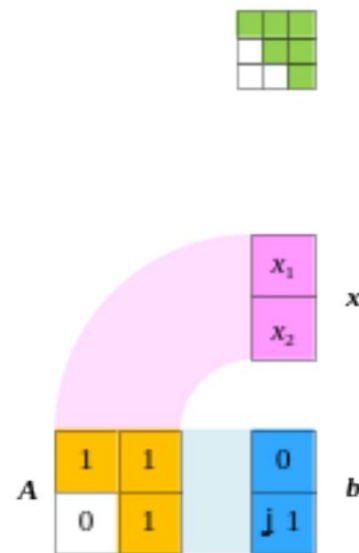
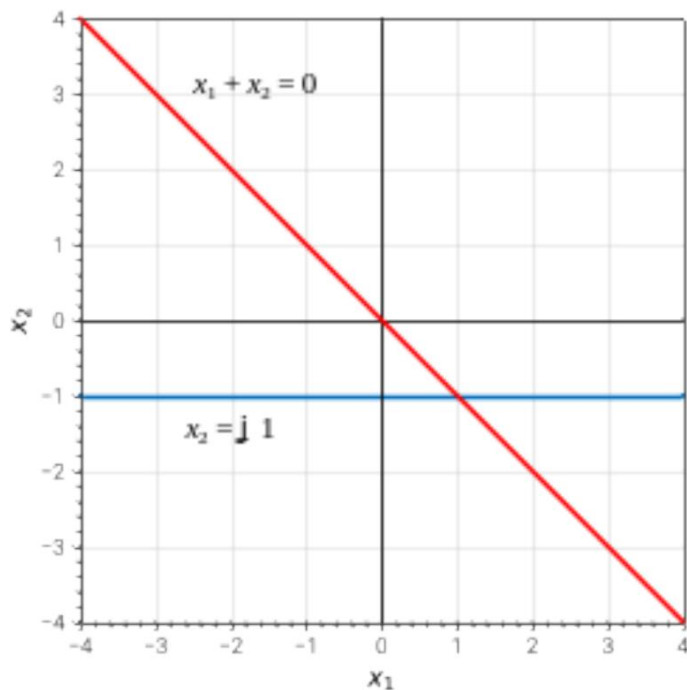


图 10. 方阵 A 是上三角矩阵

图11 · 行倍加会改变直线的形状、位置

- 记作 $R_1 - R_2 \rightarrow R_1$ ；第一行减去第二行、替换第一行，便求得 x_1 的解， A 也因此化为单位矩阵 I

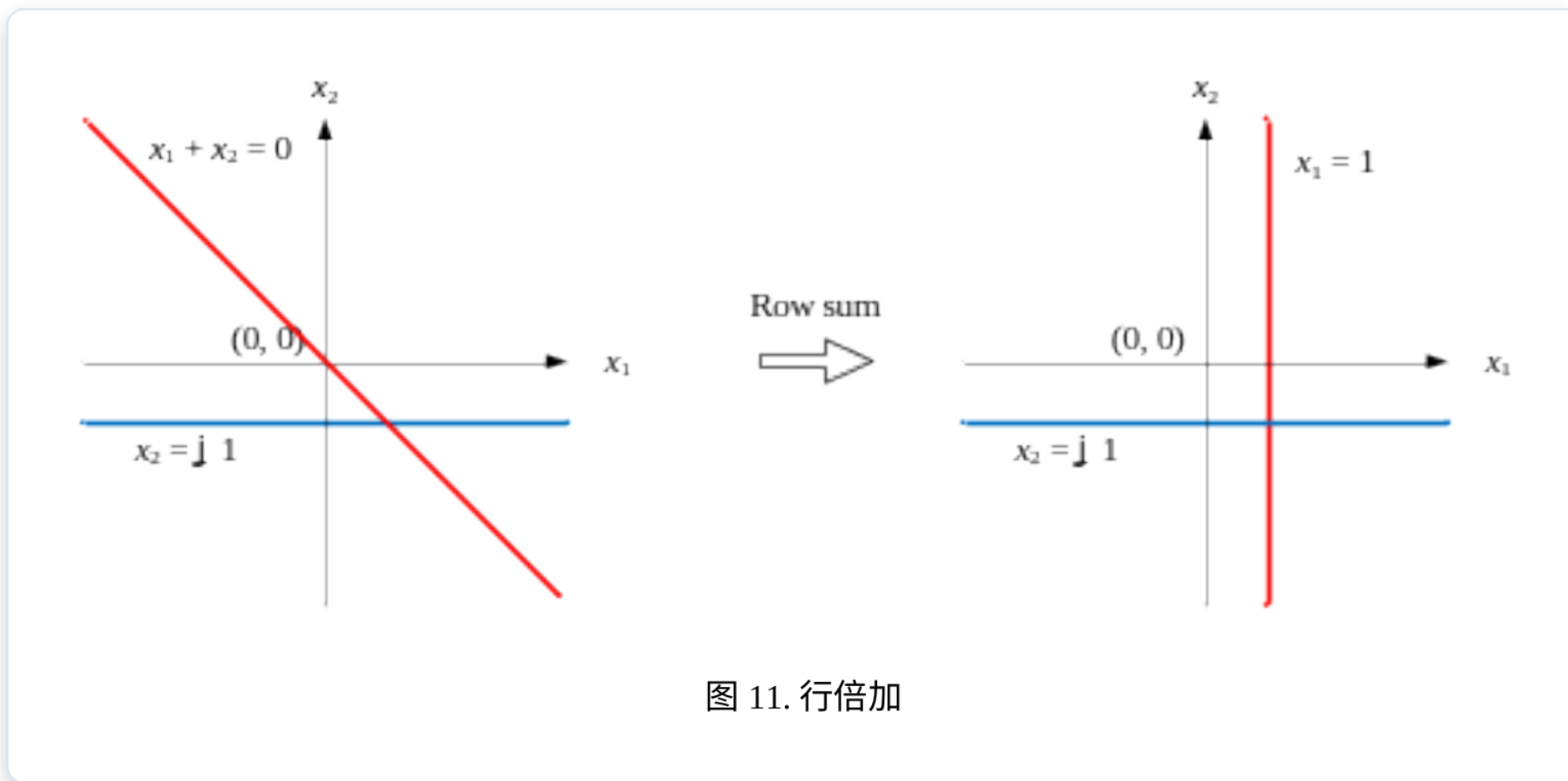


图 11. 行倍加

A 是下三角矩阵

图12 • 下三角矩阵，同理可化简

- $A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$, $\det(A) = 1$, 可逆; 记作 $R_2 - R_1 \rightarrow R_2$, 即可把 A 化为单位矩阵 I

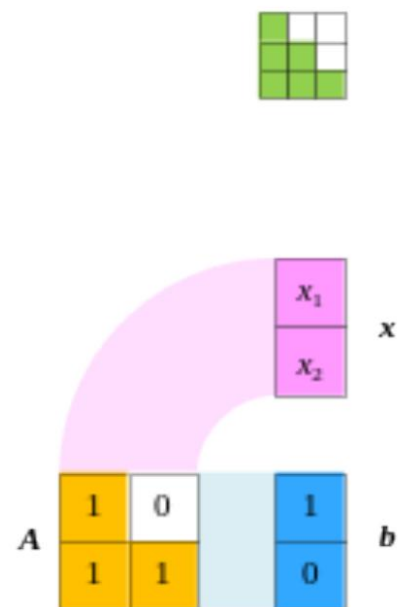
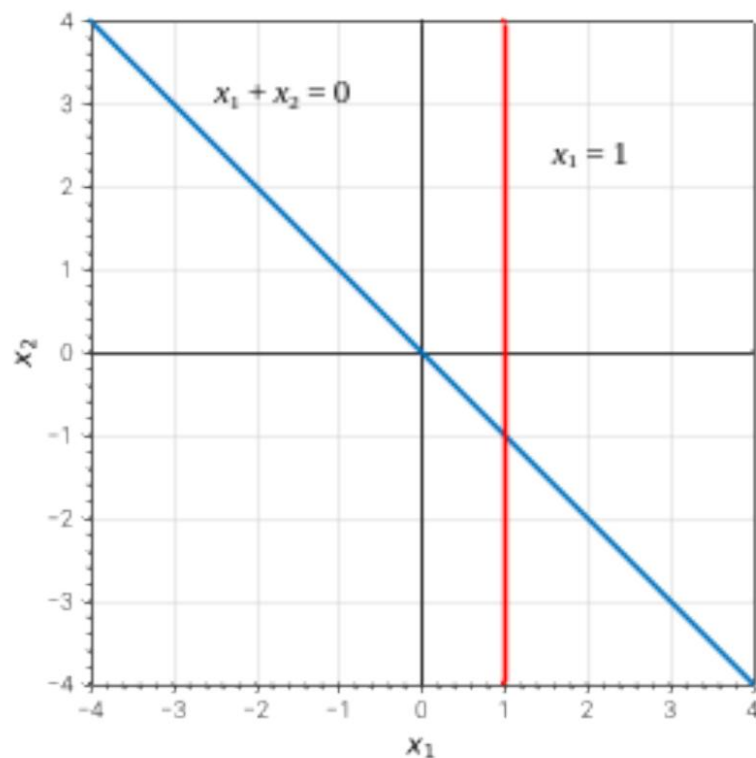


图13 · $Ax=b$ 有唯一解

- $A = \begin{bmatrix} 1 & -1 \\ 2 & 2 \end{bmatrix}$, $\det(A)=8$; 先化上（或下）三角，再化对角阵，最终化为单位阵 I

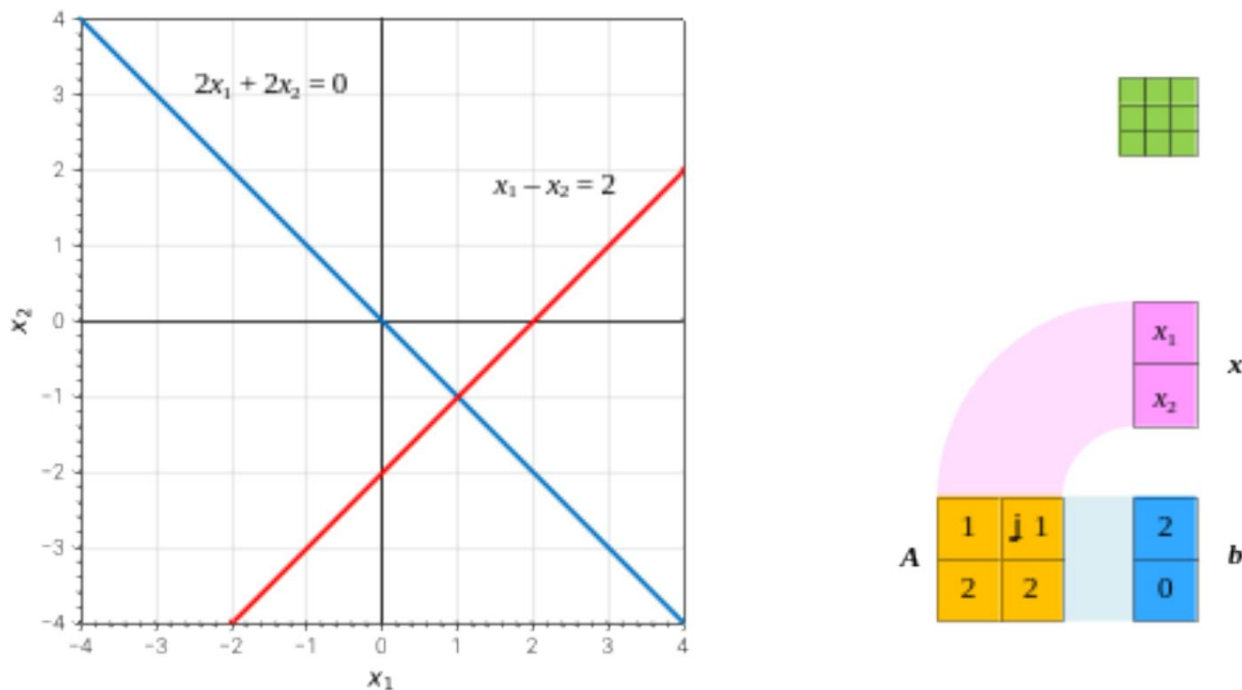


图 13. 方阵 A 行列式不为 0, $Ax=b$

常数项全为零

齐次线性方程组 (homogeneous linear system)

$$A\mathbf{x} = \mathbf{0}$$

- 常数项全部为零 ($\mathbf{b}$ 为零向量 $\mathbf{0}$) 的线性方程组, 叫齐次线性方程组
- 两条直线都经过坐标原点; 当 $\det(\mathbf{A}) \neq 0$ 时, 原点(0, 0)是唯一的解

$$AX=0$$

图14 • 两条直线均通过坐标原点

- 两条直线的交点就是齐次线性方程组的解——本例中 $\det(\mathbf{A}) \neq 0$ ，解唯一，即原点 $(0, 0)$

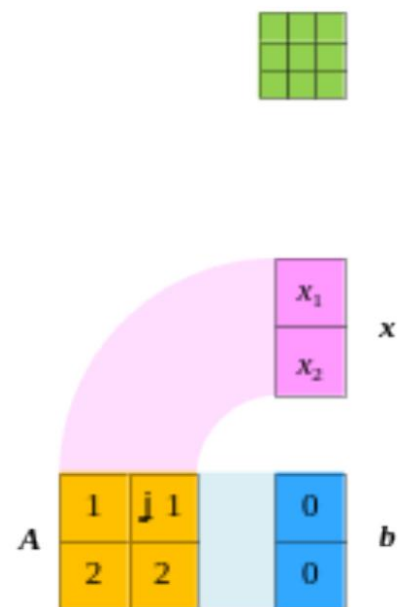
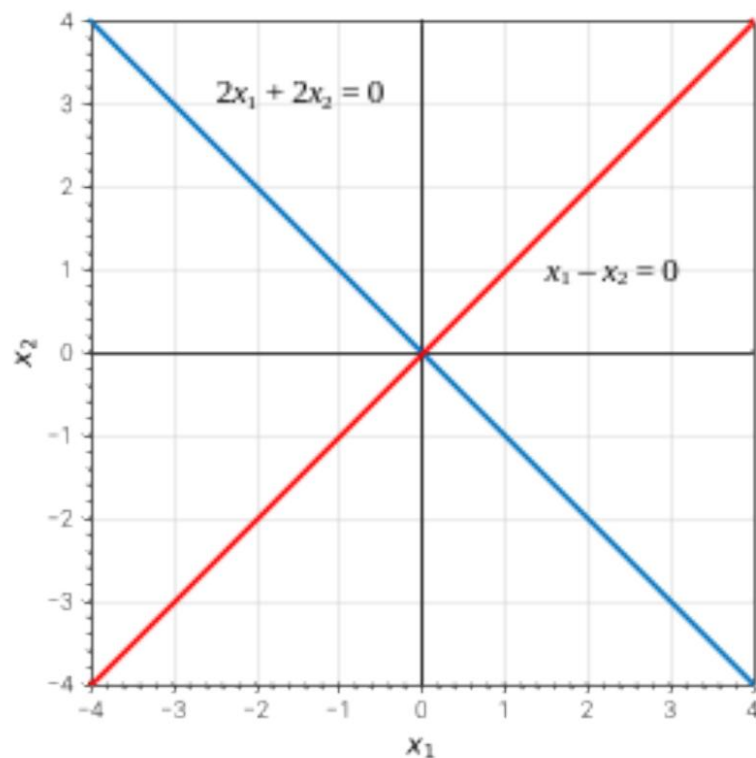


图15 · 两直线重合，无数组解

- A 的两条斜线完全重合, $\det(A)=0$, A 不存在逆, 方程组 $Ax=b$ 有无数组解

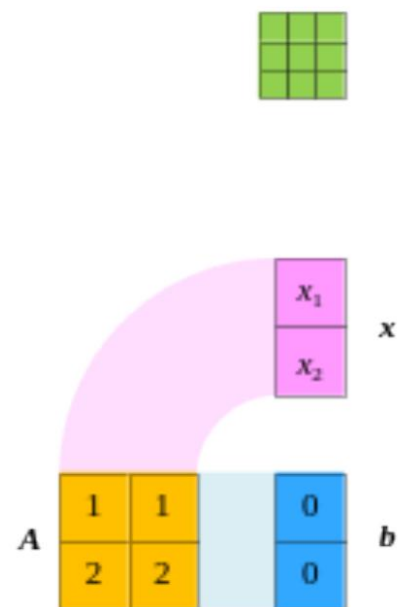
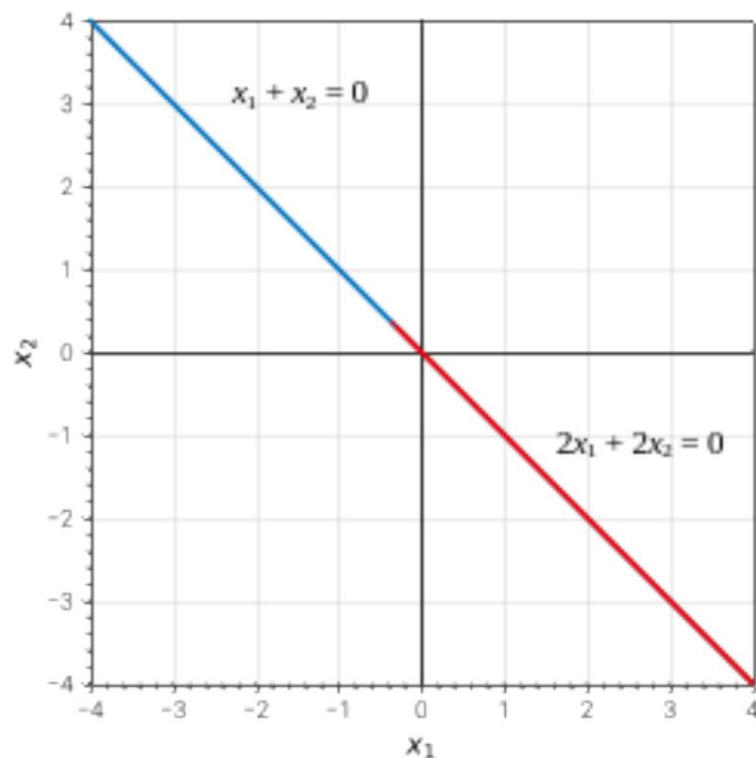
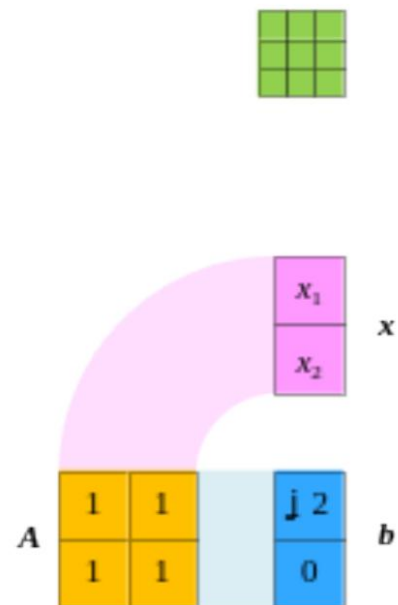
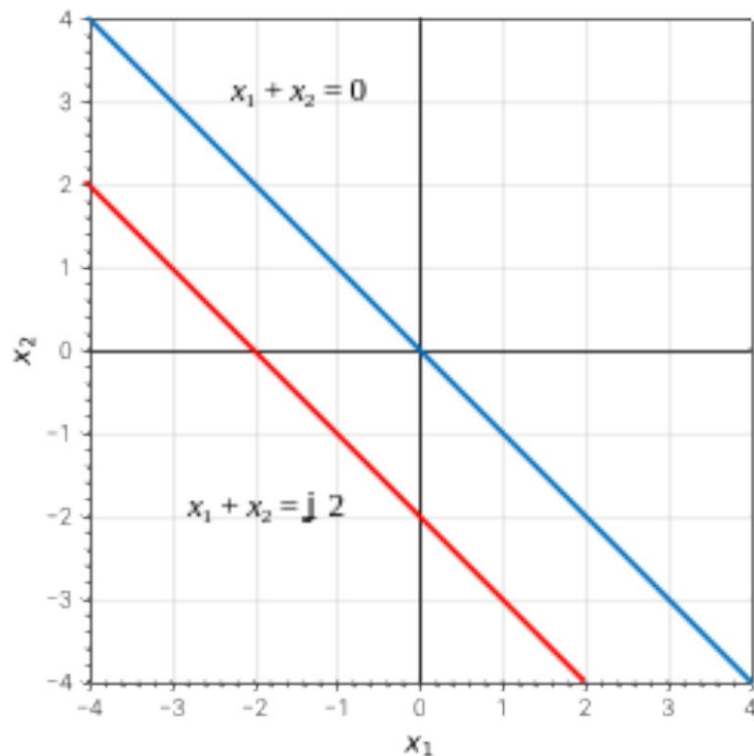


图16 · 两直线平行，无解

- A 的两条斜线相互平行， $\det(A)=0$ ， A 不存在逆，方程组 $Ax=b$ 无解



从直线交点，到矩阵与初等行变换

$\det(A) \neq 0 \Leftrightarrow$ 唯一解; $\det(A) = 0 \Leftrightarrow$ 无解或无数组解

- 求解 $Ax = b$ 的核心思路：用初等行变换（行交换、行数乘、行倍加）把 A 逐步整理成单位矩阵 I
- 行列式 $\det(A)$ 决定解的情况：不为0则唯一解，为0则无解或无数组解，需结合 b 进一步判断

本节关键术语

二元一次方程 *Linear Equation in Two Variables* ——
 $a_1x_1 + a_2x_2 = b$

系数矩阵 *Coefficient Matrix $\mathbf{A}$* —— 由方程组各未知数系数排列而成

行列式 *Determinant $\det(\mathbf{A})$* —— 判断方阵 $\mathbf{A}$ 是否可逆的关键指标

置换矩阵 *Permutation Matrix* —— 对应“行交换”操作的特殊方阵

齐次线性方程组 *Homogeneous Linear System* —— 常数项 $\mathbf{b}$ 全为零的方程组

线性方程组 *System of Linear Equations* —— 多个线性方程组成的方程组

常数列向量 *Vector of Constant Terms $\mathbf{b}$* —— 方程组等号右侧的常数

初等行变换 *Elementary Row Operations* —— 行交换、行数乘、行倍加

对角矩阵 *Diagonal Matrix* —— 只有对角线上元素非零的方阵

逆矩阵 *Inverse Matrix $\mathbf{A}^{-1}$* —— 求解 $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ 的关键工具

延伸阅读

延伸阅读 • `numpy.linalg.solve()`

LA_07_01_02.ipynb 用`solve()`直接求解鸡兔同笼问题，无需手动求逆矩阵

延伸阅读 • SymPy符号求解

LA_07_01_03.ipynb 用SymPy对线性方程组做符号化精确求解

延伸阅读 • 等高线可视觉解

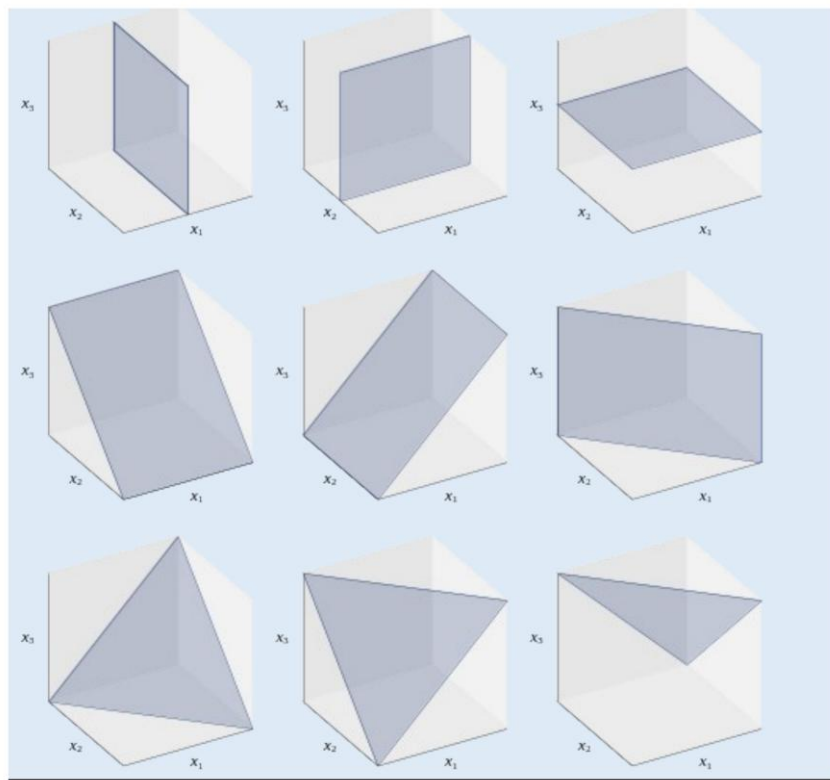
LA_07_01_04.ipynb 通过绘制等高线，可视化线性方程组的解

开源仓库

全书代码与图像素材: <https://github.com/Visualize-ML>

图17 · 9个平面示意图，配合习题Q3

- 三元一次方程 $a_1x_1 + a_2x_2 + a_3x_3 = b$ 对应三维空间中的一个平面，系数正负、是否为零决定平面的朝向



习题 Q1 – Q8

Q1 把如下两个线性方程组分别写成矩阵乘法形式： $x_1 + x_2 = 4$ ， $2x_1 + 4x_2 = 12$ ； $x_1 + x_2 = 2$ ， $x_1 - x_3 = 1$ ， $x_2 - x_3 = 0$ 。

Q3 三元一次方程对应三维空间中的一个平面；观察9幅平面示意图，判断每幅图对应方程中系数 a_1 、 a_2 、 a_3 的正负、是否为零，以及 b 的取值。

Q5 用`numpy.linalg.inv()`求Q4中可逆矩阵的逆矩阵，并用`@`验证 $A_inv@A$ 与 $A@A_inv$ 均为单位矩阵。

Q7 用`matplotlib`画出 $x_1 + x_2 = 35$ 和 $2x_1 + 4x_2 = 94$ 对应的两条直线，并标出交点坐标。

Q2 用`numpy.linalg.solve()`或逆矩阵法，编写Python代码求解Q1中的两个线性方程组。

Q4 用`numpy.linalg.det()`判断下列矩阵是否可逆： $\begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}$ 、 $\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ 、 $\begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$ 、 $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ 。

Q6 写一个函数`solve_2x2(A, b)`，用逆矩阵法求解 2×2 线性方程组，并在 $\det(A) = 0$ 时给出提示信息。

Q8 用`numpy`验证：矩阵 $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ 左乘任意二维向量，效果等价于交换该向量的两个分量。

习题 Q9 – Q16

Q9 写一个函数`row_swap(A, i, j)`，实现矩阵A第i行、第j行的行交换。

Q11 写一个函数`row_add(A, i, j, k)`，实现把第j行的k倍加到第i行（行倍加）。

Q13 用numpy验证：上三角矩阵与下三角矩阵的行列式，都等于对角线元素之积。

Q15 用numpy随机生成100个 2×2 矩阵，统计其中行列式为0（不可逆）的比例。

Q10 写一个函数`row_scale(A, i, k)`，实现将矩阵A第i行整体乘以非零标量k（行数乘）。

Q12 综合`row_swap`、`row_scale`、`row_add`三个函数，手写高斯消元，把一个 2×2 可逆矩阵化为单位矩阵。

Q14 构造一个 3×3 上三角矩阵和一个 3×3 下三角矩阵，用`numpy.linalg.det()`验证Q13的结论。

Q16 用numpy验证：齐次线性方程组 $A\mathbf{x}=\mathbf{0}$ ，当 $\det(A) \neq 0$ 时只有零解 $\mathbf{x}=[0,0]^T$ 。

习题 Q17 – Q25

Q17 构造一个 $\det(A)=0$ 的 2×2 齐次线性方程组，用`numpy.linalg.matrix_rank()`判断其解的个数是否无穷。

Q19 写一个函数`classify_2x2_system(A, b)`，根据 $\det(A)$ 以及 A 、 $[A|b]$ 的秩，自动判断方程组是唯一解、无解还是无数组解。

Q21 用`sympy.linsolve()`符号化求解鸡兔同笼问题，并与`numpy`数值解结果对比。

Q23 用`numpy.linalg.cond()`计算本节几个系数矩阵 A 的条件数，思考条件数与数值求解稳定性的关系。

Q25 综合练习：任选一个三元一次方程组，写成矩阵形式 $Ax=b$ ，并用Python判断其解的情况（唯一/无穷/无解）。

Q18 用`matplotlib`分别画出 $\det(A)=0$ 时“无穷多解”与“无解”两种情况对应的直线图（参考图15、图16）。

Q20 用Q19写的函数，测试本节图4、图13、图15、图16对应的四组方程组，验证判断结果是否正确。

Q22 改编鸡兔同笼问题：笼中有龟和鹤，共有头80个、脚236只，求龟、鹤各多少只，并用Python求解。

Q24 写一个函数`plot_2x2_system(A, b)`，自动绘制任意 2×2 线性方程组对应的两条直线及交点（若存在）。

10个可以和AI互动的Prompt

- 1 请解释为什么二元一次方程的图像是一条直线，而三元一次方程的图像是一个平面？
- 2 请证明：当方阵 A 的行列式 $\det(A)=0$ 时，线性方程组 $Ax=b$ 要么无解，要么有无数组解。
- 3 请解释初等行变换中“行交换”“行数乘”“行倍加”分别对应哪些特殊矩阵？
- 4 请用Python演示：用初等行变换把一个一般的 2×2 可逆矩阵，一步步化简为单位矩阵。
- 5 请解释为什么单位矩阵是求解 $Ax=b$ 的“终极目标”？

10个可以和AI互动的Prompt

6

请解释齐次线性方程组 $Ax=0$ 与非齐次方程组 $Ax=b$ 的解之间有什么关系？

7

请举一个现实生活中可以转化为二元一次方程组的例子，并把它写成矩阵形式。

8

请解释为什么工程和数据科学中偏好用矩阵形式 $Ax=b$ ，而不是逐个方程求解？

9

请用NumPy对比numpy.linalg.inv()和numpy.linalg.solve()两种方法求解同一线性方程组，比较计算效率。

10

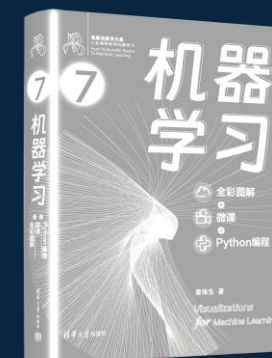
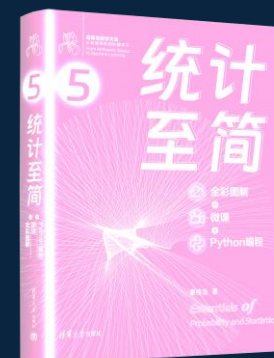
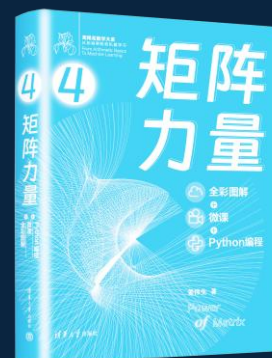
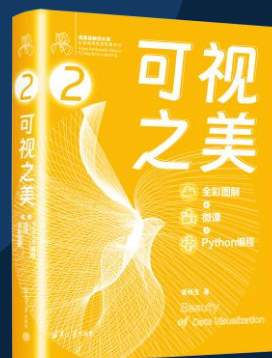
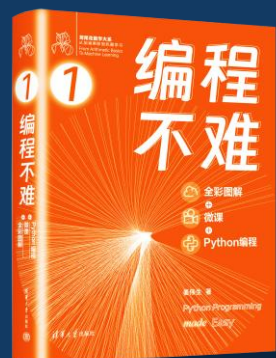
请设计一个Python小项目：输入任意 2×2 线性方程组的系数，自动判断解的情况并可视化两条直线。

数学

数学基础

线性代数

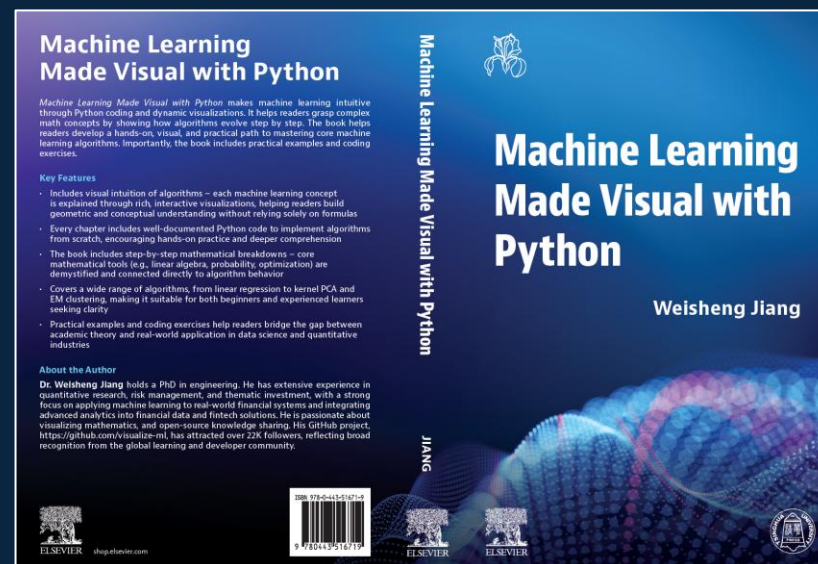
概率统计



Python编程

数据可视化

工具



数据分析
图和网络

回归、降维
分类、聚类

实践